# Relearning Containers 09: How Containerd stores container images and data

June 2, 2021 | 6752 reads

Previously we briefly understood the architecture of Containerd. This section will take a look at how Containerd stores images and containers, including how to store images and RootFS.

## From pulling the image to starting the container

There are two configurations in the Containerd configuration file:

```
root = /var/lib/containerd
state = "/run/containerd"
```

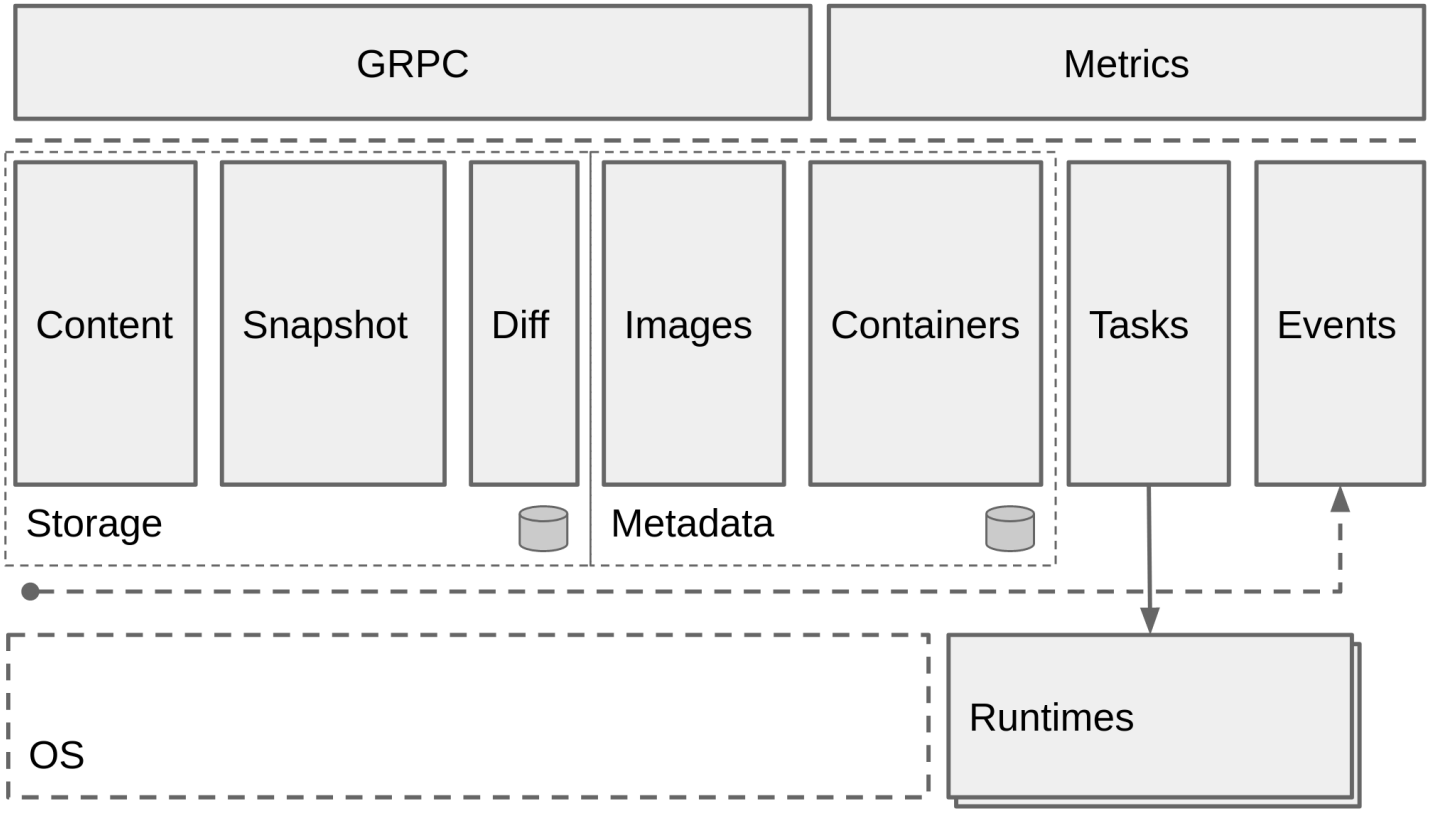
The directory configured by root is the directory used to save persistent data, including `content`, `snapshot`, `metadata` and `runtime`. Next, on a test server, after deleting all images and containers, execute the following command to reinitialize these directories to prepare for subsequent experiments.

```
systemctl stop containerd
rm -rf /var/lib/containerd/*
rm -rf /var/lib/nerdctl/*
systemctl start containerd
```

After completing the initialization of the test environment, check `/var/lib/containerd` the directory again:

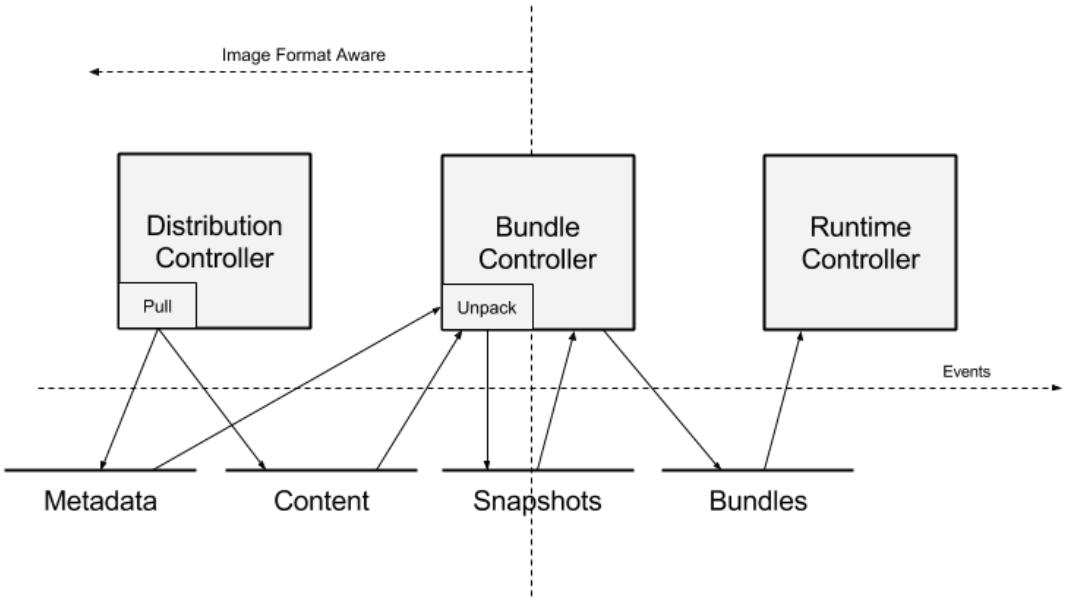
```
tree /var/lib/containerd/ -L 2
/var/lib/containerd/
├── io.containerd.content.v1.content
│   └── ingest
├── io.containerd.metadata.v1.bolt
│   └── meta.db
├── io.containerd.runtime.v1.linux
├── io.containerd.runtime.v2.task
├── io.containerd.snapshotter.v1.btrfs
├── io.containerd.snapshotter.v1.native
│   └── snapshots
├── io.containerd.snapshotter.v1.overlayfs
│   └── snapshots
└── tmpmounts
```

`/var/lib/containerd` `content` Each subdirectory of , clearly corresponds to the subsystems and components in the architecture diagram of `snapshot`, , `metadata` and containerd: `runtime`




`/var/lib/containerd` The names of each subdirectory below can also correspond to `ctr plugin ls` the names of some plug-ins used to view and print. In fact, these directories are the directories used by Containerd plug-ins to save data. Each plug-in can have its own separate data directory, and Containerd itself does not store data. , all its functions are implemented through plug-ins.

According to the data flow diagram of containerd below, follow the steps of pulling the image, starting the container, creating a file in the container, and observe the changes in the data directory of containerd.




First pull an image:

```
nerdctl pull redis:alpine3.13
docker.io/library/redis:alpine3.13:                                       resolved       |++++++++++++++++++++++++++++++++++++++|
index-sha256:eaaa58f8757d6f04b2e34ace57a71d79f8468053c198f5758fd2068ac235f303:    done           |++++++++++++++++++++++++++++++++++++++|
manifest-sha256:b7cb70118c9729f8dc019187a441980418a87e6a837f4846e87130df379e2c8: done           |++++++++++++++++++++++++++++++++++++++|
```

```
config-sha256:1690b63e207f6651429bebd716ace700be29d0110a0cfefff5038bb2a7fb6fc7:   done           |++++++++++++++++++++++++++++++++++++++|
layer-sha256:6ab1d05b49730290d3c287ccd34640610423d198e84552a4c2a4e98a46680cfd:    done           |++++++++++++++++++++++++++++++++++++++|
layer-sha256:8cc52074f78e0a2fd174bdd470029cf287b7366bf1b8d3c1f92e2aa8789b92ae:    done           |++++++++++++++++++++++++++++++++++++++|
layer-sha256:aa7854465cce07929842cb49fc92f659de8a559cf521fc7ea8e1b781606b85cd:    done           |++++++++++++++++++++++++++++++++++++++|
layer-sha256:8173c12df40f1578a7b2dfbbc0034a4fbc8ec7c870fd32b9236c2e5e1936616a:    done           |++++++++++++++++++++++++++++++++++++++|
layer-sha256:540db60ca9383eac9e418f78490994d0af424aab7bf6d0e47ac8ed4e2e9bcbba:    done           |++++++++++++++++++++++++++++++++++++++|
layer-sha256:29712d301e8c43bcd4a36da8a8297d5ff7f68c3d4c3f7113244ff03675fa5e9c:    done           |++++++++++++++++++++++++++++++++++++++|
elapsed: 16.4s                                                                    total:  7.7 Mi (481.5 KiB/s)
```

Judging from the execution process of the above command, a total of 1 index, 1 config, 1 manifest and 6 layers are pulled.

`io.containerd.metadata.v1.bolt/meta.db` It is a boltdb file that stores persistent references to images and bundles. boltdb is an embedded key/value database. The comments in the header of containerd's source code file https://github.com/containerd/containerd/blob/master/metadata/buckets.go describe the db schema data structure.

```
// keys.
//  ├──version : <varint>                       - Latest version, see migrations
//  └──v1                                       - Schema version bucket
//     └══*namespace*
//        ├──labels
//        │  └══*key* : <string>                 - Label value
//        ├──image
//        │  └══*image name*
//        │     ├──createdat : <binary time>     - Created at
//        │     ├──updatedat : <binary time>     - Updated at
//        │     ├──target
//        │     │  ├──digest : <digest>          - Descriptor digest
//        │     │  ├──mediatype : <string>       - Descriptor media type
//        │     │  └──size : <varint>            - Descriptor size
//        │     └──labels
//        │        └══*key* : <string>           - Label value
//        ├──containers
//        │  └══*container id*
//        │     ├──createdat : <binary time>     - Created at
//        │     ├──updatedat : <binary time>     - Updated at
//        │     ├──spec : <binary>               - Proto marshaled spec
//        │     ├──image : <string>              - Image name
//        │     ├──snapshotter : <string>        - Snapshotter name
//        │     ├──snapshotKey : <string>        - Snapshot key
//        │     ├──runtime
//        │     │  ├──name : <string>            - Runtime name
//        │     │  ├──extensions
//        │     │  │  └══*name* : <binary>       - Proto marshaled extension
//        │     │  └──options : <binary>         - Proto marshaled options
//        │     └──labels
//        │        └══*key* : <string>           - Label value
//        ├──snapshots
//        │  └══*snapshotter*
//        │     └══*snapshot key*
//        │        ├──name : <string>            - Snapshot name in backend
//        │        ├──createdat : <binary time>  - Created at
//        │        ├──updatedat : <binary time>  - Updated at
//        │        ├──parent : <string>          - Parent snapshot name
//        │        ├──children
//        │        │  └══*snapshot key* : <nil>  - Child snapshot reference
```

```
//        |        └──labels
//        |           └══*key* : <string>        - Label value
//        ├──content
//        |  ├──blob
//        |  |  └══*blob digest*
//        |  |     ├──createdat : <binary time>  - Created at
//        |  |     ├──updatedat : <binary time>  - Updated at
//        |  |     ├──size : <varint>            - Blob size
//        |  |     └──labels
//        |  |        └══*key* : <string>        - Label value
//        |  └──ingests
//        |     └══*ingest reference*
//        |        ├──ref : <string>             - Ingest reference in backend
//        |        ├──expireat : <binary time>   - Time to expire ingest
//        |        └──expected : <digest>        - Expected commit digest
//        └──leases
//           └══*lease id*
//              ├──createdat : <binary time>     - Created at
//              ├──labels
//              |  └══*key* : <string>           - Label value
//              ├──snapshots
//              |  └══*snapshotter*
//              |     └══*snapshot key* : <nil>  - Snapshot reference
//              ├──content
//              |  └══*blob digest* : <nil>      - Content blob reference
//              └──ingests
//                 └══*ingest reference* : <nil> - Content ingest reference
```

It can be seen that metadata about images, content, snapshots, and containers is mainly recorded. A simple go program is written here to read and print the contents of the current boltdb. Note that some binary types in the above structural description will be printed as garbled characters.

```go
package main

import (
    "fmt"
    "log"

    bolt "go.etcd.io/bbolt"
)
func main() {
    {
        db, err := bolt.Open("/var/lib/continaerd/io.containerd.metadata.v1.bolt/meta.db", 0666, nil)
        if err != nil {
            log.Fatal(err)
        }
        defer db.Close()
        db.View(func(tx *bolt.Tx) error {
            b := tx.Bucket([]byte("v1")).Bucket([]byte("default"))
            space := ""
            travelBucket(b, space)
            return nil
        })
    }

}
func travelBucket(b *bolt.Bucket, space string) {
```

```
    space = space + "\t"
    b.ForEach(func(k, v []byte) error {
        if v == nil {
            fmt.Printf("%sbucket=%s: \n", space, k)
            travelBucket(b.Bucket([]byte(k)), space)
        } else {
            fmt.Printf("%skey=%s, value=%s\n", space, k, v)
        }
        return nil
    })

}
```

All you need to understand here is that meta.db stores the metadata of each storage. So where are the actual pull images stored? The image content is stored in `io.containerd.content.v1.content/blobs/sha256`:

```
ll io.containerd.content.v1.content/blobs/sha256
total 10668
1690b63e207f6651429bebd716ace700be29d0110a0cfefff5038bb2a7fb6fc7
29712d301e8c43bcd4a36da8a8297d5ff7f68c3d4c3f7113244ff03675fa5e9c
540db60ca9383eac9e418f78490994d0af424aab7bf6d0e47ac8ed4e2e9bcbba
6ab1d05b49730290d3c287ccd34640610423d198e84552a4c2a4e98a46680cfd
8173c12df40f1578a7b2dfbbc0034a4fbc8ec7c870fd32b9236c2e5e1936616a
8cc52074f78e0a2fd174bdd470029cf287b7366bf1b8d3c1f92e2aa8789b92ae
aa7854465cce07929842cb49fc92f659de8a559cf521fc7ea8e1b781606b85cd
b7cb70118c9729f8dc019187a4411980418a87e6a837f4846e87130df379e2c8
eaaa58f8757d6f04b2e34ace57a71d79f8468053c198f5758fd2068ac235f303
```

The above 9 files correspond to 1 index file, 1 config, 1 manifest file and 6 layer files. The index and manifest can be viewed directly with the cat command, and the layer file can be decompressed with tar. Therefore, the config, manifest, and tar files saved in the content are the set of OCI image standards.

In fact, containerd does decompress the tar in these contents into snapshots.

View `/var/lib/containerd` catalog:

```
tree /var/lib/containerd/ -L 3
/var/lib/containerd/
├── io.containerd.content.v1.content
│   ├── blobs
│   │   └── sha256
│   └── ingest
├── io.containerd.metadata.v1.bolt
│   └── meta.db
├── io.containerd.runtime.v1.linux
├── io.containerd.runtime.v2.task
├── io.containerd.snapshotter.v1.btrfs
├── io.containerd.snapshotter.v1.native
│   └── snapshots
├── io.containerd.snapshotter.v1.overlayfs
│   ├── metadata.db
│   └── snapshots
│       ├── 1
│       ├── 2
│       ├── 3
│       ├── 4
│       ├── 5
```

```
└── 6
└── tmpmounts
```

You can see `io.containerd.snapshotter.v1.overlayfs/snapshots` that there are 6 more subdirectories named 1~6. Check these 6 directories:

```
tree /var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/ -L 3
/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/
├── 1
│   ├── fs
│   │   ├── bin
│   │   ├── dev
│   │   ├── etc
│   │   ├── home
│   │   ├── lib
│   │   ├── media
│   │   ├── mnt
│   │   ├── opt
│   │   ├── proc
│   │   ├── root
│   │   ├── run
│   │   ├── sbin
│   │   ├── srv
│   │   ├── sys
│   │   ├── tmp
│   │   ├── usr
│   │   └── var
│   └── work
├── 2
│   ├── fs
│   │   ├── etc
│   │   └── home
│   └── work
├── 3
│   ├── fs
│   │   ├── etc
│   │   ├── lib
│   │   ├── sbin
│   │   ├── usr
│   │   └── var
│   └── work
├── 4
│   ├── fs
│   │   ├── bin
│   │   ├── etc
│   │   ├── lib
│   │   ├── tmp
│   │   ├── usr
│   │   └── var
│   └── work
├── 5
│   ├── fs
│   │   └── data
│   └── work
└── 6
    ├── fs
    │   └── usr
```

```
52      └── work
```

The main function of containerd's snapshotter is to prepare rootfs for the container by mounting each layer. The default snapshotter configured by containerd is overlayfs, which is an implementation of a joint file system. Overlayfs turns the read-only image layer into lowerdir, the read-write container layer into upperdir, and finally the combined mount presents mergedir.

Start a redis container below:

```
1 nerdctl run -d --name redis redis:alpine3.13
```

You can see `io.containerd.snapshotter.v1.overlayfs/snapshots` that there is an additional directory named 7:

```
1  tree /var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/ -L 2
2  /var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/
3  ├── 1
4  │   ├── fs
5  │   └── work
6  ├── 2
7  │   ├── fs
8  │   └── work
9  ├── 3
10 │   ├── fs
11 │   └── work
12 ├── 4
13 │   ├── fs
14 │   └── work
15 ├── 5
16 │   ├── fs
17 │   └── work
18 ├── 6
19 │   ├── fs
20 │   └── work
21 └── 7
22     ├── fs
23     └── work
```

You can use the mount command to view the RootFS of the overlayfs mounted by the container:

```
1 mount | grep /var/lib/containerd
2 overlay on /run/containerd/io.containerd.runtime.v2.task/default/8102f7fbee26792830e54e80b3488714ac559e092c59beb2e311cf8e
```

It can be seen that: `snapshots/6/fs` , `snapshots/5/fs` ... `snapshots/1/fs` is lowerdir and `snapshots/7/fs` is upperdir. The directory finally presented by joint mounting and merging is `/run/containerd/io.containerd.runtime.v2.task/default/8102f7fbee26792830e54e80b3488714ac559e092c59beb2e311cf8e88f475d6/rootfs` the rootfs of the container. If you look at this directory with ls, you can see a typical Linux system directory structure:

```
1 ls /run/containerd/io.containerd.runtime.v2.task/default/8102f7fbee26792830e54e80b3488714ac559e092c59beb2e311cf8e88f475d6/
2 bin  data  dev  etc  home  lib  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
```

We exec into the container and create a hello file in the /root directory:

```
1 nerdctl exec -it redis sh
2 echo hello > /root/hello
```



This file can be found in the upperdir on the host machine. Changes to the file system in the container will be reflected in upperdir:

```
ls /var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/7/fs/root/
hello
```

## containerd的Snapshotter

Finally, let's take a look at containerd's Snapshot component. Snapshot implements Snapshotter for containerd to manage snapshots of container images on the file system and rootfs mounting and unmounting of containers. Snapshotter is designed to benchmark the graphdriver storage driver in Docker. contaienrd uses the new snapshotter model in design to replace the graphdriver in docker. Its core developers also introduced why they did this in their blog Where are containerd's graph drivers?

## reference


- https://github.com/containerd/containerd
- https://github.com/etcd-io/bbolt
- Where are containerd's graph drivers?
- Contaienrd Snapshots


# related articles

- Re-learning Containers 08: Briefly understand the architecture of Containerd
- Relearning Containers 07: The technology behind container isolation namespace
- Relearning Containers 06: The Technology Behind Container Resource Limitations cgroups
- Relearning containers 05: Use nerdctl + buildkitd to build container images
- Relearning containers 04: containerd's high-level command line tool nerdctl

containerd


ALSO ON 青蛙小白

使用kubeadm部署 Kubernetes 1.22
2 年前 · 4条评论

rust语言基础学习: 使用 AsRef和AsMut ...
2 年前 · 1条评论
昨天我们学习了Default trait，通过实现Default ...

Kubernetes ...
5 年前 · 1条评论

使用kubeadm部署 Kubernetes 1.27
5 个月前 · 1条评论
使用kubeadm部署 Kubernetes 1.27

使用kubeadm安 Kubernetes 1.15
4 年前 · 11条评论

**1条评论** 登录 ▾

G 加入讨论...

通过以下方式登录 或注册一个 DISQUS 帐号 ?

姓名

♡ 分享 最佳 最新 最早

M **min wang**
3 个月前
大佬，您的文章写的是真的好，循序渐进、由浅入深，牛！
0 0 回复 • 分享 ›

订阅 隐私 不要出售我的数据

© 2023frogxiaobai By frognew | This site is powerfully driven by Hugo , and the theme is transplanted from maupassant